

Auto-Scaling Web Infrastructure on AWS

Terraform · Python · AWS · ap-south-1 Mumbai

1. Project Overview

This project provisions, configures, and monitors a fully automated, production-style web infrastructure on Amazon Web Services — built entirely using Infrastructure as Code (IaC). Nothing is created manually through the AWS Console. Every resource is defined in code, version-controlled in Git, and reproducible with a single command.

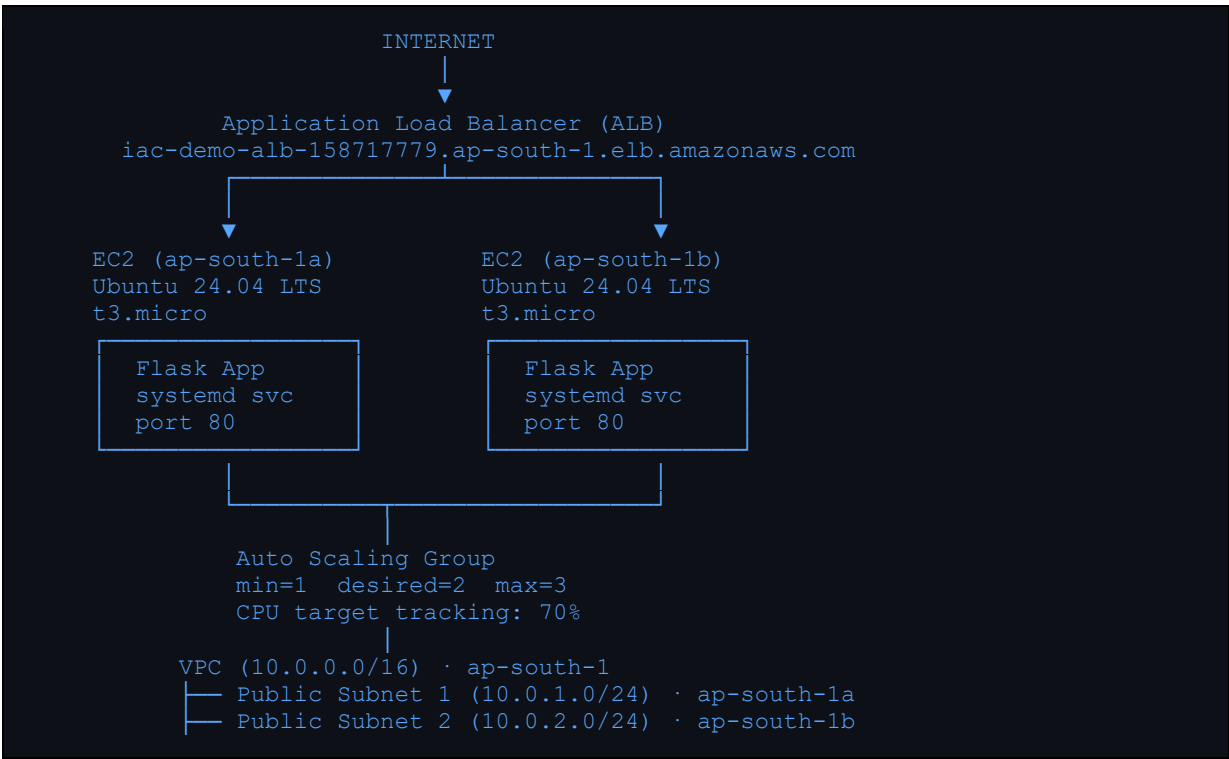
A Flask web application runs across multiple EC2 instances behind an Application Load Balancer that distributes traffic automatically. The Auto Scaling Group adds instances when CPU exceeds 70% and removes them when load drops — without human intervention.

Every new EC2 instance bootstraps itself completely on boot via cloud-init userdata — installing Python, Flask, creating a systemd service, and starting the app automatically. The ALB verifies each instance passes a health check before sending it traffic.

What makes this a real project, not a tutorial

This infrastructure was built from scratch and encountered real-world problems that had to be diagnosed and solved — wrong AMI, SSH key mismatches, port binding permissions, ASG lifecycle design, IAM credential architecture, and more. Each problem is documented in Section 8.

2. Architecture



```
└─ Internet Gateway
└─ Route Table (0.0.0.0/0 → IGW)
└─ ALB Security Group (:80 from 0.0.0.0/0)
└─ EC2 Security Group (:80 from ALB only, :22 SSH)
```

3. Tech Stack

Tool	Version	Role
Terraform	v1.16.1	Provision all 15 AWS resources as Infrastructure as Code
Ansible	2.16.3	Configuration management — playbook for server setup
Python + Flask	3.12 + 3.1.3	Web application running on each EC2 instance
boto3	1.43.87	AWS SDK — infrastructure health monitoring script
AWS EC2	t3.micro	Compute — Ubuntu 24.04 LTS (ami-0f58b397bc5c1f2e8)
AWS ALB	—	Application Load Balancer — traffic distribution + health checks
AWS ASG	—	Auto Scaling Group — manages instance count based on CPU
AWS VPC	—	Isolated network across 2 availability zones in Mumbai

4. Development Environment

Final architecture

```
Windows 11
└─ WSL2 Ubuntu 24.04
    └─ Terraform
    └─ Ansible
    └─ AWS CLI
    └─ Python
    └─ boto3
        ↓
        AWS ap-south-1
        └─ Infrastructure
```

Why WSL2?

Terraform was originally intended to run natively on Windows. However, during development the Windows Terraform execution caused problems, and the project was successfully continued using Terraform inside WSL2 Ubuntu. Therefore, the currently verified working environment is **WSL2 Ubuntu**, not native Windows Terraform.

Ansible does not support Windows as a control node — it requires a Linux/Unix environment. WSL2 (Windows Subsystem for Linux) provides a full Ubuntu environment inside Windows 11 where Ansible, AWS CLI, and Python tooling run natively without any workarounds.

Why a Python virtual environment?

Ubuntu 24.04 enforces PEP 668 — it blocks pip installs to the system Python to prevent dependency conflicts. A virtual environment (aws-env) isolates project dependencies cleanly without using `--break-system-packages`.

IAM setup

A dedicated IAM user (terraform-user) was created with AdministratorAccess for this learning project. The AWS CLI profile "terraform" was configured with its access key. The root account was not used for development — a security best practice.

```
AWS Root Account
├─ IAM User: terraform-user
│   ├─ Policy: AdministratorAccess
│   ├─ AWS CLI profile: terraform
│   └─ Used by: Terraform, AWS CLI, boto3
```

5. How It Works — Layer by Layer

Layer 1 — Terraform provisions infrastructure

Terraform reads the .tf configuration files and makes API calls to AWS to create all resources. It maintains a state file that tracks what exists — running terraform apply twice does not create duplicate resources. Terraform compares desired state (your files) against current state and makes only the minimum necessary changes.

- ▶ **terraform init** — downloads the AWS provider plugin (~5.0)
- ▶ **terraform plan** — previews what will be created. Always review before apply.
- ▶ **terraform apply** — creates all 15 resources in AWS. Takes ~2 minutes.
- ▶ **terraform destroy** — deletes all resources cleanly. Always run when done.

Layer 2 — userdata.sh bootstraps every EC2 instance

The Launch Template embeds userdata.sh as a base64-encoded script. When the ASG launches any new EC2 instance, AWS runs this script automatically via cloud-init on first boot. This is the key design decision that makes new instances self-configuring.

What userdata.sh does on every new instance:

- apt-get update and install python3, python3-pip, python3-venv
- Create /home/ubuntu/app directory
- Create Python virtual environment at /home/ubuntu/app/venv
- Install Flask inside the venv
- Write app.py directly to /home/ubuntu/app/app.py
- Create /etc/systemd/system/flaskapp.service (User=root for port 80 binding)
- systemctl daemon-reload, enable flaskapp, start flaskapp

Result: every new ASG instance becomes fully configured and serves traffic without any manual SSH or Ansible intervention.

Layer 3 — Ansible for configuration management

An Ansible playbook was written and executed to deploy and configure the Flask application on existing EC2 instances. The playbook uses the following tasks: verify Python3 and venv, create app directory, create virtual environment, install Flask, copy app.py, create systemd service, and enable/start it.

Design decision: for ephemeral ASG instances, userdata is the correct bootstrap mechanism because Ansible would need a dynamic inventory system and an automated trigger to run on every new instance. Ansible is retained in the project to demonstrate configuration management knowledge — it is the correct tool for managing long-lived server fleets.

Layer 4 — Flask application

A lightweight Python web application runs on each EC2 instance as a systemd service. It exposes two routes:

- ▶ `/` — returns an HTML page showing the server hostname and current timestamp. Refreshing the browser causes the ALB to route to a different instance, changing the hostname — proving load balancing works.
- ▶ `/health` — returns `{"status": "healthy", "host": "hostname"}` with HTTP 200. The ALB calls this endpoint every 30 seconds to verify the instance is alive.

Important: Flask was run as root (User=root in systemd) because Linux blocks non-root processes from binding to ports below 1024. Port 80 requires root.

Layer 5 — ALB health checks and self-healing

The ALB calls `/health` on every registered instance every 30 seconds. Two consecutive successful responses (HTTP 200) = healthy, instance receives traffic. Two consecutive failures = unhealthy, removed from rotation. The ASG detects the unhealthy instance and launches a replacement using the Launch Template. The replacement bootstraps itself via userdata and registers with the ALB automatically.

Layer 6 — boto3 health monitoring

A Python script queries three AWS APIs simultaneously and prints a live dashboard:

- ▶ **autoscaling client** — queries `DescribeAutoScalingGroups` to get instance IDs, lifecycle state (InService/Pending/Terminating), and ASG health status
- ▶ **elbv2 client** — queries `DescribeTargetHealth` to get ALB view of each instance (healthy/unhealthy/drainning)
- ▶ **ec2 client** — queries `DescribeInstances` to get public IP, private IP, instance type, and launch time

boto3 is the official AWS SDK for Python. Every boto3 call maps directly to an AWS CLI command — it is AWS CLI in Python, enabling automation and integration into monitoring pipelines.

6. AWS Resources Created (15 total)

Resource	Name	Purpose
aws_vpc	iac-demo-vpc	Isolated network, CIDR 10.0.0.0/16, 65,536 IPs
aws_internet_gateway	iac-demo-igw	Connects VPC to public internet
aws_subnet ×2	public-subnet-1/2	One per AZ — ap-south-1a (10.0.1.0/24), ap-south-1b (10.0.2.0/24)
aws_route_table	iac-demo-public-rt	Routes 0.0.0.0/0 traffic to Internet Gateway

aws_route_table_association ×2	—	Links each subnet to the route table
aws_security_group (ALB)	iac-demo-alb-sg	Allows inbound TCP 80 from 0.0.0.0/0
aws_security_group (EC2)	iac-demo-ec2-sg	Allows TCP 80 from ALB SG only, TCP 22 SSH
aws_lb	iac-demo-alb	Public-facing Application Load Balancer across both subnets
aws_lb_target_group	iac-demo-tg	Pool of EC2 targets, /health check, HTTP 200 expected
aws_lb_listener	—	Listens on port 80, forwards to target group
aws_launch_template	iac-demo-lt	EC2 blueprint: Ubuntu AMI, t3.micro, userdata, key pair
aws_autoscaling_group	iac-demo-asg	min=1, desired=2, max=3, across both subnets
aws_autoscaling_policy	iac-demo-cpu-policy	TargetTrackingScaling — CPU target 70%

7. Key Concepts — Explained for Interviews

Infrastructure as Code (IaC)

IaC means defining cloud infrastructure in machine-readable configuration files instead of manually clicking through a console. Every resource — VPC, subnets, load balancer, EC2 instances — is declared in .tf files. The entire infrastructure is reproducible with two commands (terraform destroy + terraform apply) and every change is version-controlled in Git.

Why it matters: eliminates environment drift, enables team collaboration, makes infrastructure auditable, and allows disaster recovery by recreating everything from code.

Idempotency

Running the same operation multiple times produces the same result. Terraform compares desired state (.tf files) against current state (.tfstate) and only makes necessary changes — running terraform apply twice does not create duplicate resources. Ansible playbooks are also idempotent — installing a package that is already installed does nothing.

Why it matters: automation is safe to re-run. Critical for CI/CD pipelines where the same deployment runs on every code push.

High Availability

Instances are spread across two Availability Zones (ap-south-1a and ap-south-1b). Availability Zones are physically separate data centres in the same region — distinct power, cooling, and networking. The ALB also spans both subnets. If one AZ has an outage, traffic automatically continues to instances in the other AZ.

Auto Scaling

The Auto Scaling Group monitors average CPU utilisation via CloudWatch. When CPU exceeds 70% (TargetTrackingScaling policy), AWS automatically launches new EC2 instances from the Launch Template and registers them with the ALB. When CPU drops, instances are terminated. No human action required.

Self-healing infrastructure

If an EC2 instance fails its ALB health check (/health returning non-200), the ALB stops routing traffic to it. The ASG detects the unhealthy instance and launches a replacement automatically. The replacement bootstraps itself via userdata and becomes healthy without any intervention.

Separation of concerns

Each tool has a single, clear responsibility: Terraform provisions infrastructure, userdata bootstraps instances, Ansible manages configuration, Python monitors the running system. This makes each layer independently testable, maintainable, and replaceable.

Declarative vs imperative

Terraform is declarative — you describe what you want, not how to build it. You write "I want a VPC with CIDR 10.0.0.0/16" and Terraform figures out the API calls. Scripts are imperative — you write step-by-step instructions. Declarative is easier to reason about and less error-prone.

8. Real Challenges Solved

This project was not a tutorial walkthrough. These are real problems that were encountered, diagnosed, and fixed.

Challenge 1 — Wrong AMI / OS mismatch

Original config used Amazon Linux 2 with yum package manager and ec2-user SSH username. The actual AMI (ami-0f58b397bc5c1f2e8) was Ubuntu 24.04 LTS which uses apt and ubuntu user. All scripts, userdata, and Ansible tasks were rewritten for Ubuntu.

Challenge 2 — Instance type rejection

t2.micro was rejected as not free-tier eligible on this AWS account. Used `aws ec2 describe-instance-types --filters "Name=free-tier-eligible,Values=true"` to identify t3.micro as the correct eligible type.

Challenge 3 — Port 80 permission denied

Flask running as ubuntu user crashed with "Permission denied" — Linux blocks non-root processes from binding to ports below 1024. Diagnosed via `sudo journalctl -u flaskapp`. Fixed by setting `User=root` in the systemd service unit file.

Challenge 4 — SSH key mismatch

Two key pairs (iac-demo-key, iac-demo-key-v2) did not match the local .pem file. Fixed by creating iac-demo-key-v3, verifying the public key with `ssh-keygen -y -f` against the AWS Console public key, and updating Terraform variable.tf.

Challenge 5 — ASG instance cycling

With `desired_capacity=2`, the ASG kept replacing the second instance because it was unhealthy (Flask not running). The root cause was that userdata only installed prerequisites but did not deploy the app. Fixed by embedding the complete app deployment — app.py, systemd service, and service start — directly into userdata.sh.

Challenge 6 — AWS credentials for Terraform

`aws login --remote` produced temporary root credentials that Terraform could not reliably consume. Fixed by creating a dedicated IAM user (terraform-user) with a permanent access key and configuring an AWS CLI profile named "terraform".

Challenge 7 — boto3 unhashable type error

Flask /health route returned a raw Python dict, causing `TypeError: unhashable type: dict` in some Flask versions. Fixed by using `jsonify()` from flask to properly serialise the response to JSON before returning.

Challenge 8 — Terraform desired_capacity override

AWS CLI changes to desired_capacity were overridden when terraform apply ran, reverting to the value in main.tf. Fixed by updating desired_capacity directly in main.tf and applying — Terraform is the single source of truth for infrastructure state.

9. Project Structure

```
aws-iac-project/
├── terraform/
│   ├── main.tf           # All 15 AWS resources
│   ├── variable.tf       # Region, AMI, instance type, key pair name
│   ├── outputs.tf        # ALB DNS, ASG name, VPC ID, Target Group ARN
│   └── userdata.sh        # EC2 bootstrap — self-configuring on boot
├── ansible/
│   ├── inventory.ini     # EC2 public IPs + SSH connection vars
│   └── playbook.yml      # Flask deploy: venv, app, systemd, service
├── script/
│   └── health_check.py   # boto3 — ASG + ALB + EC2 health dashboard
├── app.py                # Flask app: / and /health routes
├── .gitignore            # Excludes tfstate, credentials, venv
└── README.md             # Full project documentation
```

10. GitHub Repository Setup

Repository details

URL: github.com/vijayJG/aws-iac-asg.tf

Repo description (one line under repo name)

Auto-scaling AWS infrastructure with Terraform, Ansible, and Python. VPC · ALB · ASG · EC2 · IaC · ap-south-1

Topics / tags to add

terraform ansible aws infrastructure-as-code devops python boto3

ec2 auto-scaling cloud flask vpc alb iac ap-south-1